DBSCAN, PCA & t-SNE

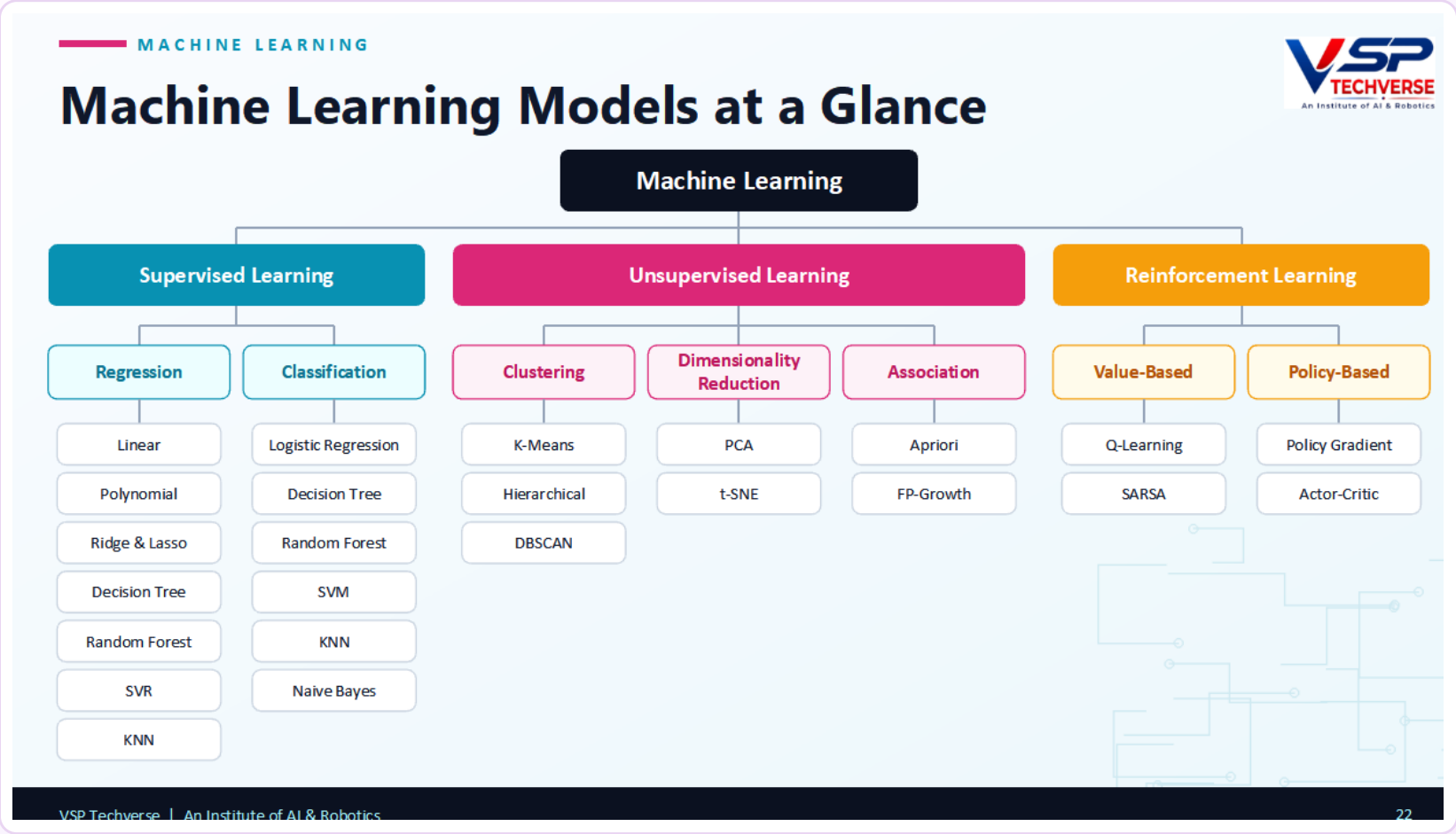


Finding groups and shrinking big data, explained step by step for AI & Robotics

VSP TECHVERSE
An Institute of AI & Robotics

REFERENCE

Machine Learning Models at a Glance



FIND IT ON THE MAP

Clustering

→ **DBSCAN**

FIND IT ON THE MAP

Dimensionality reduction

→ **PCA and t-SNE**

REMEMBER Today we live in the Unsupervised branch.

RECAP

Unsupervised Learning: No Answers Given

In supervised learning every row has an answer (label). In unsupervised learning there is no answer column. The computer looks for patterns by itself.

Clustering

Put similar rows into groups. Example: group LiDAR points into objects. Today: DBSCAN.

Dimensionality reduction

Many columns → few columns, keeping the important story. Today: PCA and t-SNE.

Why robots need it

Robots collect lots of unlabelled data (sensors, scans, motion). We must make sense of it first.

WORD CHECK A column = one feature (e.g. temperature). A row = one observation (e.g. one robot reading). Dimensions = number of columns.

DATA

Today's Robot Datasets

DBSCAN

robot_lidar_scan.csv

315 points

INPUTS

x_m, y_m (where the laser hit something)

ANSWER COLUMN

no labels

PCA

robot_sensor_readings.csv

300 readings

INPUTS

8 sensor columns (temperature, vibration, ...)

ANSWER COLUMN

status: normal / warning / fault

t-SNE

robot_gesture_data.csv

400 gestures

INPUTS

12 motion-sensor columns

ANSWER COLUMN

gesture: 4 hand gestures

PART 1

DBSCAN

Finding crowds and ignoring loners

- Everyday idea: towns seen at night
- eps, min_samples, core / border / noise
- Code on a robot LiDAR scan



The Idea: Towns Seen from a Plane at Night

Many lights close together

A crowd of lights = a town. In data we call this a cluster.

A town can be any shape

Towns follow roads and rivers: long, curved, bent. DBSCAN can follow any shape.

A lone light far away

One farm light is not a town. DBSCAN calls it noise and ignores it.

DBSCAN = Density-Based Spatial Clustering of Applications with Noise. Simple meaning: "find the crowded places".

ROBOTICS EXAMPLE A LiDAR scan: points on a wall or box form crowds (objects); a stray reflection is noise.

Two Settings Control Everything

eps (epsilon)

The radius of the circle drawn around each point. Think: how far can I reach with my arm?

```
eps = 0.35
```

min_samples

How many points must be inside that circle to call it a crowd. Think: how many people make a crowd?

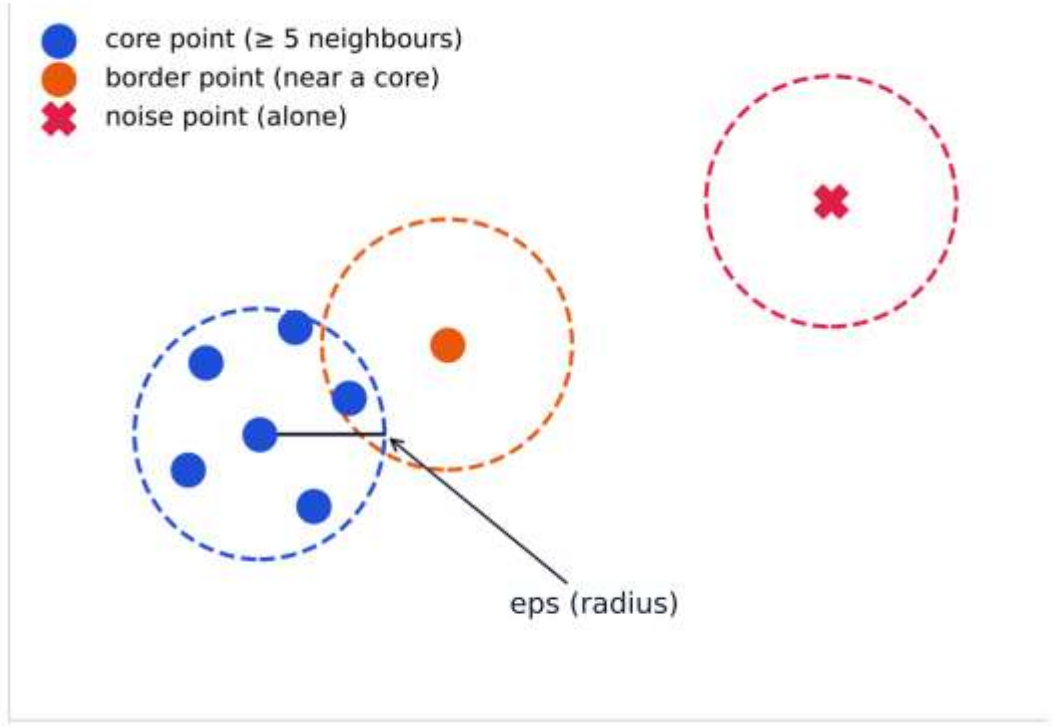
```
min_samples = 5
```

Too small eps → a crowd breaks into many tiny groups and lots of noise.

Too big eps → separate towns merge into one big blob.

Tip: we usually start with min_samples = 5 and tune eps.

Three Kinds of Points



Core point
Has at least min_samples points inside its eps circle. The heart of a crowd.

Border point
Inside a core point's circle but not crowded itself. The edge of the crowd.

Noise point
Not near any core point. Gets label -1 and belongs to no cluster.

REMEMBER Clusters are made of core and border points. Noise (-1) is a feature, not a bug: it tells the robot "ignore these".

DBSCAN in 4 Steps

1 Pick a point

Start with any point that has not been visited yet.

2 Count neighbours

Count points inside its eps circle. Enough (≥ min_samples)? It is a core point.

3 Grow the cluster

Add every neighbour, then check their neighbours too. Keep spreading.

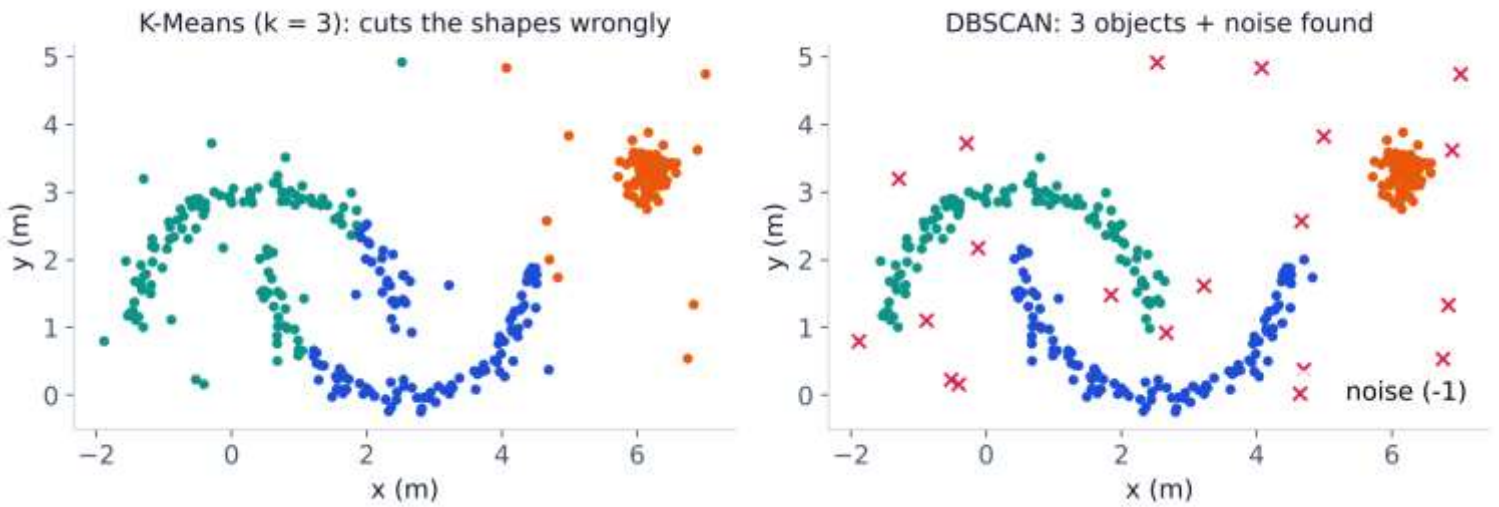
4 Repeat

Stop when the crowd ends. Pick a new unvisited point. Leftovers = noise.

THINK OF IT AS Like a rumour spreading through a crowd: it passes from neighbour to neighbour until it reaches people who have no one nearby.

You never tell DBSCAN how many clusters to find. It discovers the number by itself.

Why Not Just Use K-Means?



- K-Means draws round blobs. It cuts curved shapes in the wrong place
- K-Means forces every point into a group, even stray ones
- K-Means needs k in advance
- DBSCAN follows the real shape and sets noise aside

ROBOTICS EXAMPLE A corridor wall in a LiDAR scan is long and thin, not round. K-Means would split it; DBSCAN keeps it as one object.

Code: Finding Objects in a LiDAR Scan

dbscan.py

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN

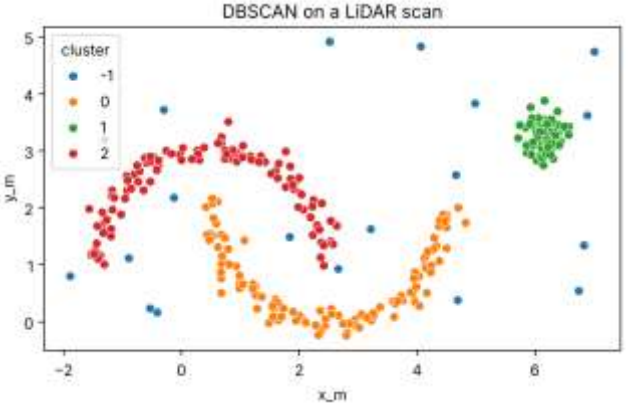
# 1. Load LiDAR scan points (no labels)
df = pd.read_csv("robot_lidar_scan.csv")
X = df[["x_m", "y_m"]]

# 2. DBSCAN: eps = neighbourhood radius, min_samples = crowd size
model = DBSCAN(eps=0.35, min_samples=5)
df["cluster"] = model.fit_predict(X)

# 3. -1 means NOISE (a point that belongs to no object)
print("Objects found:", df["cluster"].max() + 1)
print("Noise points :", (df["cluster"] == -1).sum())
print(df["cluster"].value_counts().sort_index())

# 4. Plot every object in its own colour
sns.scatterplot(data=df, x="x_m", y="y_m", hue="cluster",
                palette="tab10")
plt.title("DBSCAN on a LiDAR scan")
plt.show()
```

RESULT



OUTPUT

```
Objects found: 3
Noise points : 19
cluster
-1     19
 0    115
 1     70
 2    111
Name: count, dtype: int64
```

DBSCAN found 3 objects and set 19 stray points aside as noise (-1).

How Do We Choose eps?

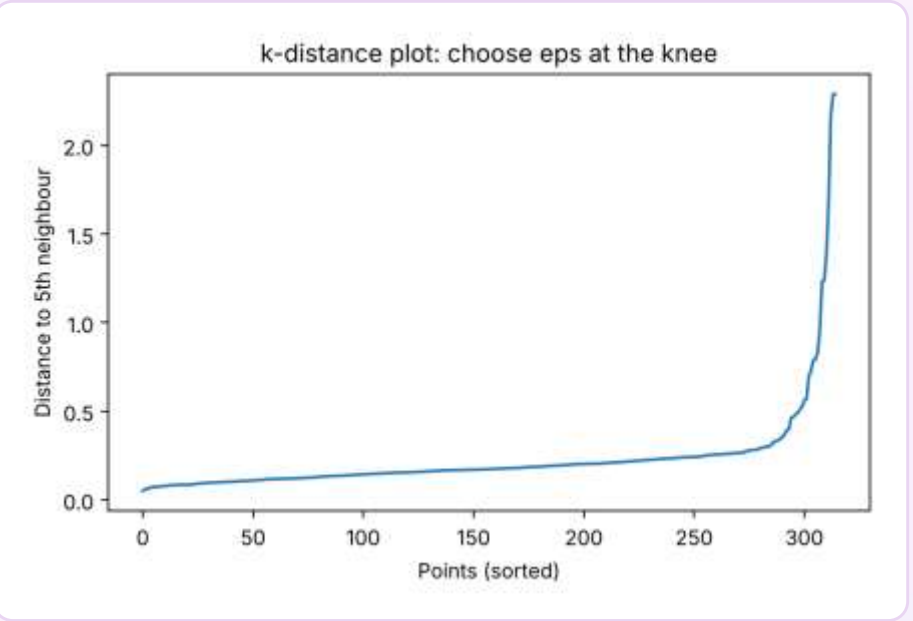
k_distance.py

```
from sklearn.neighbors import NearestNeighbors

# distance from every point to its 5th nearest neighbour
nn = NearestNeighbors(n_neighbors=5).fit(X)
distances, _ = nn.kneighbors(X)
k_dist = sorted(distances[:, 4])

plt.plot(k_dist)
plt.xlabel("Points (sorted)"); plt.ylabel("Distance to 5th
neighbour")
plt.title("k-distance plot: choose eps at the knee")
plt.show()
```

K-DISTANCE PLOT



Look for the "knee" where the line bends sharply upward. The height at the knee (about 0.35) is a good eps.

DBSCAN: Good & Bad

ADVANTAGES

- No need to choose the number of clusters
- Finds clusters of any shape
- Detects noise and outliers automatically
- Not disturbed by a few stray points

DISADVANTAGES

- You must tune eps and min_samples
- Struggles when clusters have very different densities
- Needs scaling when columns use different units
- Slower on very large, high-dimensional data

BEST FIT IN ROBOTICS Finding obstacles in LiDAR scans, grouping GPS stops, spotting odd sensor readings.

PART 2

PCA

Shrinking many columns into a few

- Why too many columns is a problem
- Best camera angle: the main idea
- Scaling, components, explained variance



The Problem: Too Many Columns

Cannot be drawn

We can plot 2 or 3 columns. A robot with 8 sensors lives in 8 dimensions. We cannot see that.

Slow to process

More columns mean more calculation and more memory on a small robot computer.

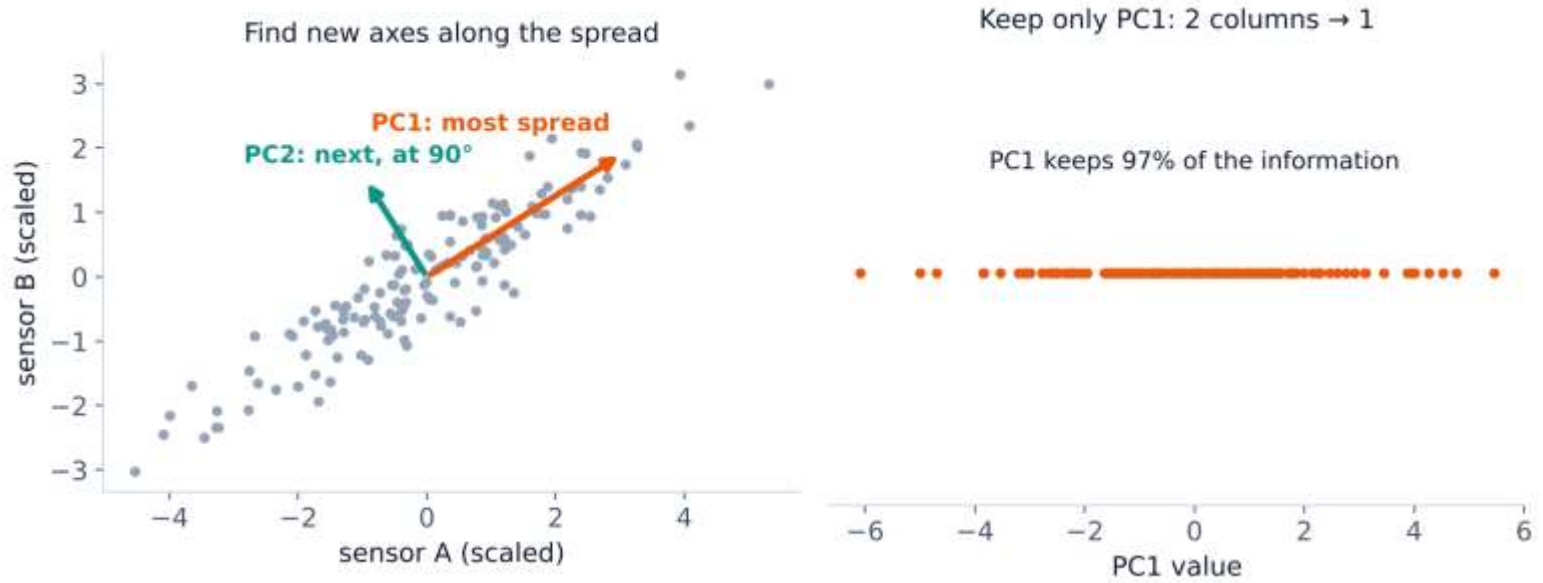
Repeated information

Temperature, current and wear often rise together. Several columns tell the same story.

Goal: turn 8 columns into 2 new columns that still keep most of the information.

PCA does this by mixing the old columns into a few smart new ones, called principal components (PC1, PC2, ...).

The Idea: Find the Best Camera Angle



- A 3D statue photographed from the front looks flat and boring
- Photographed from the best angle, you see its whole shape
- PCA turns the data to find the angle with the most spread
- PC1 = direction with the most spread. PC2 = the next best direction

KEY IDEA Spread = information. If robots all have the same value in a column, that column tells us nothing. Directions where robots differ most are the most useful.

PCA in 4 Steps

1 Scale

Put every column on the same scale (mean 0, spread 1).

2 Find directions

Search for the directions where the data is most spread out.

3 Sort them

PC1 = most spread, PC2 = second most, and so on.

4 Keep the top few

Keep PC1, PC2 and drop the rest. Project the data onto them.

EXAMPLE Result: 300 readings × 8 columns → 300 readings × 2 columns, ready to plot.

PCA is unsupervised: it never looks at the status column. It only studies the sensors.

Why Scale First?

Imagine two sensors with very different units. Without scaling, the big numbers win unfairly.

Motor temperature

20 to 90 °C

Large numbers → looks very important

Vibration

0.1 to 2 mm/s

Small numbers → looks unimportant

`StandardScaler`

gives every column mean 0 and spread 1, so every sensor gets a fair vote.

THE FORMULA

$$\text{scaled value} = (\text{value} - \text{mean}) \,/\, \text{spread}$$

GOLDEN RULE Always scale before PCA, t-SNE and DBSCAN when columns use different units.

Code: 8 Sensors Down to 2

pca.py

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

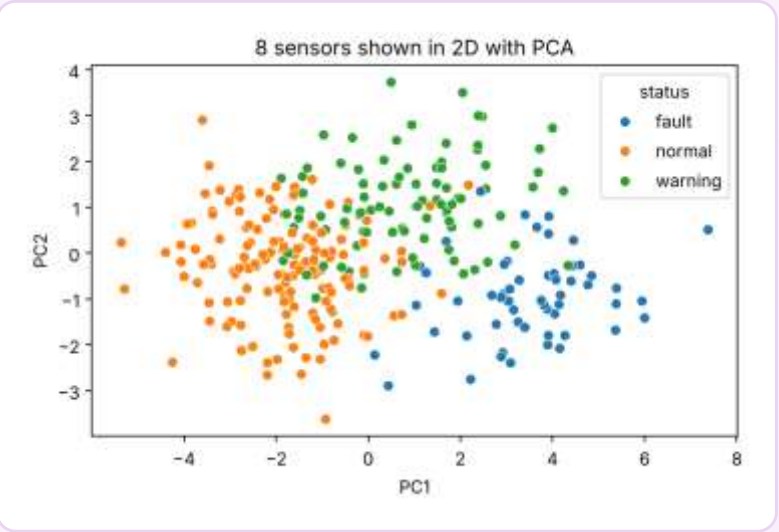
# 1. Load 8 sensor columns
df = pd.read_csv("robot_sensor_readings.csv")
X = df.drop(columns="status")

# 2. Scale: every sensor gets mean 0, std 1
X_scaled = StandardScaler().fit_transform(X)

# 3. PCA: squeeze 8 columns into 2
pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_scaled)
print("Shape before:", X.shape, " after:", X_2d.shape)
print("Variance kept by PC1, PC2:",
      pca.explained_variance_ratio_.round(3))
print("Total kept:", pca.explained_variance_ratio_.sum().round(3))

# 4. Plot the robots on the 2 new axes
df["PC1"], df["PC2"] = X_2d[:, 0], X_2d[:, 1]
sns.scatterplot(data=df, x="PC1", y="PC2", hue="status")
plt.title("8 sensors shown in 2D with PCA")
plt.show()
```

RESULT



OUTPUT

```
Shape before: (300, 8)  after: (300, 2)
Variance kept by PC1, PC2: [0.759 0.198]
Total kept: 0.957
```

Two components keep 95.7% of the information. Normal, warning and fault robots separate nicely.

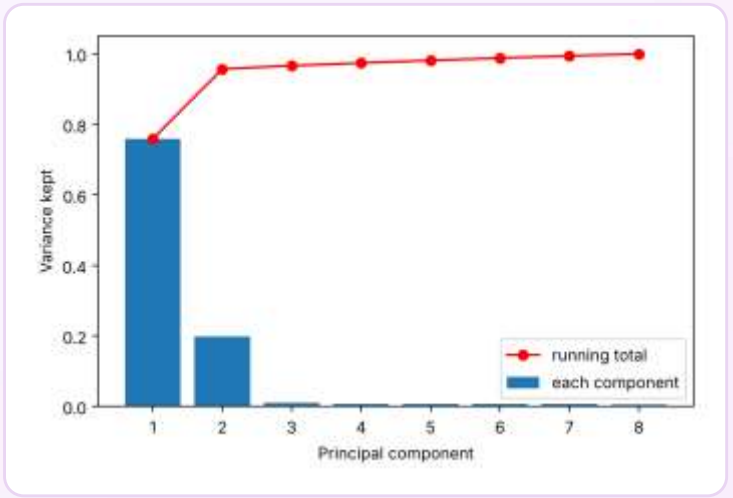
How Many Components to Keep?

scree_plot.py

```
# How much information does each component keep?
pca_all = PCA().fit(X_scaled)
ratio = pca_all.explained_variance_ratio_
print("Each PC :", ratio.round(3))
print("Running :", ratio.cumsum().round(3))

plt.bar(range(1, 9), ratio, label="each component")
plt.plot(range(1, 9), ratio.cumsum(), marker="o", color="red",
         label="running total")
plt.xlabel("Principal component"); plt.ylabel("Variance kept")
plt.legend()
plt.show()
```

SCREE PLOT



OUTPUT

```
Each PC : [0.759 0.198 0.009 0.008 0.007 0.007 0.006 0.006]
Running : [0.759 0.957 0.967 0.974 0.982 0.988 0.994 1.   ]
```

Bars drop sharply after PC2 and the red line is nearly flat: 2 components are enough.

PCA: Good & Bad

ADVANTAGES

- Fast and simple
- Fewer columns, faster models
- Removes repeated information and some noise
- Lets us plot high-dimensional data

DISADVANTAGES

- New columns are hard to explain in plain words
- Only finds straight-line patterns
- Needs scaling first
- Some information is always lost

BEST FIT IN ROBOTICS Compressing many sensor readings, speeding up robot vision features, visualising robot health data.

PART 3

t-SNE

Drawing a friendly map of your data

- Seating friends at a party
- perplexity and random_state
- PCA versus t-SNE, and how to read plots



The Idea: Seat Friends Together

The party

You have 400 guests (data points) and a big hall. Each guest has a few close friends.

The seating plan

You must seat everyone on a flat 2D floor so that friends sit close together.

The result

Groups of friends form clear tables. Different groups sit apart.

t-SNE = t-distributed Stochastic Neighbour Embedding. Simple meaning: "a 2D map where neighbours stay neighbours".

ROBOTICS EXAMPLE Robot gestures that feel similar in 12 sensor columns should land close together on the map.

How t-SNE Works

1 Find neighbours

In the original many-column data, measure who is close to whom.

2 Start with a random map

Place all points randomly on a 2D sheet.

3 Nudge points

Move points step by step: friends closer, strangers further.

IMPORTANT It keeps LOCAL neighbourhoods right. It does not promise that the distances between far-away groups mean anything.

Because it starts from a random map, results can change from run to run, unless we fix random_state.

Settings You Should Know

perplexity

Roughly "how many close friends each point has". Usually 5 to 50.

```
perplexity=30
```

Small: many tiny groups. Large: blurrier, bigger picture.

random_state

Fixes the random starting map, so you get the same picture every time.

```
random_state=42
```

Any number works. Use one for repeatable results.

n_components

How many output columns. Use 2 for plotting.

```
n_components=2
```

We nearly always use 2 with t-SNE.

Code: PCA and t-SNE Side by Side

tsne.py

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

# 1. Load 12 IMU features for 400 hand gestures
df = pd.read_csv("robot_gesture_data.csv")
X = StandardScaler().fit_transform(df.drop(columns="gesture"))

# 2. PCA to 2D (straight-line method)
pca_2d = PCA(n_components=2).fit_transform(X)

# 3. t-SNE to 2D (keeps neighbours together)
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
tsne_2d = tsne.fit_transform(X)
print("t-SNE output shape:", tsne_2d.shape)

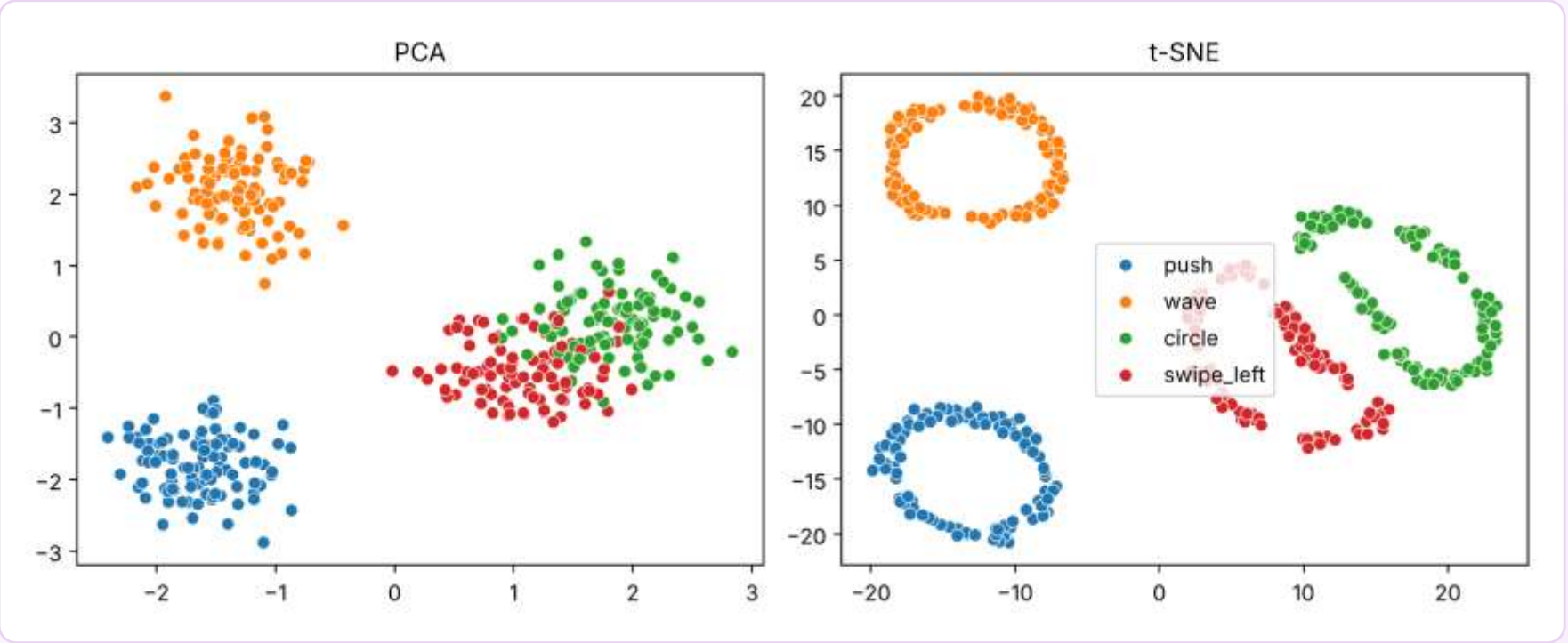
# 4. Compare the two pictures side by side
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
sns.scatterplot(x=pca_2d[:, 0], y=pca_2d[:, 1], hue=df["gesture"],
                ax=ax[0], legend=False)
ax[0].set_title("PCA")
sns.scatterplot(x=tsne_2d[:, 0], y=tsne_2d[:, 1], hue=df["gesture"],
                ax=ax[1])
ax[1].set_title("t-SNE"); ax[1].legend(loc="center")
plt.show()
```

OUTPUT

```
t-SNE output shape: (400, 2)
```

The output is a table of 400 points with 2 columns, ready to plot. The pictures come next.

Result: PCA vs t-SNE on Hand Gestures



WHY Gestures are curved patterns, so a straight-line method (PCA) mixes them, while t-SNE untangles them.

PCA: two gestures (swipe_left, circle) overlap. **t-SNE:** all four gestures form clear groups.

Careful: How to Read a t-SNE Plot

You CAN say

Points close together are similar. Clear islands are real groups.

You CANNOT say

Distance between islands means "very different". Island size means "big group".

Always remember

Try different perplexity values. Fix random_state. Use it to look, not to train models.

RULE OF THUMB If a t-SNE plot shows groups, you can trust that they exist. If it shows no groups, try another perplexity before deciding that there are none.

t-SNE: Good & Bad

ADVANTAGES

- Beautiful, clear pictures of groups
- Handles curved, tangled patterns
- Great for exploring new data
- Often reveals groups PCA misses

DISADVANTAGES

- Slower than PCA on big data
- Results change with perplexity and random_state
- Distances between groups are not reliable
- Cannot transform new data easily

BEST FIT IN ROBOTICS Exploring camera or gesture features, checking whether robot behaviours form natural groups.

DBSCAN vs PCA vs t-SNE

	DBSCAN	PCA	t-SNE
Job	Clustering	Dimensionality reduction	Dimensionality reduction
Needs labels?	No	No	No
Main idea	Find crowded areas	Rotate to the best angle	Keep neighbours together
Key settings	eps, min_samples	n_components	perplexity, random_state
Scale first?	Yes	Yes	Yes
Speed	Fast	Very fast	Slow on big data
Our result	3 objects, 19 noise	8 → 2 columns, 95.7% kept	4 gestures separated

Key Words to Remember

eps
Radius of the circle around each point

min_samples
Points needed in the circle to make a crowd

Core / border / noise
Heart of a crowd / edge / loner (label -1)

Dimension
Another word for a column or feature

Principal component
A new column made by PCA (PC1, PC2, ...)

Explained variance
How much information a component keeps

Scaling
Putting all columns on the same scale

Perplexity
Roughly how many close neighbours per point

random_state
Fixes the random start for repeatable results

Unsupervised
Learning without an answer column

PRACTICE

Try It Yourself

1

DBSCAN

Change eps to 0.15 and then 0.8. How many objects and noise points do you get? Why?

2

PCA

Use n_components=3. How much variance is kept now? Remove StandardScaler: what changes?

3

t-SNE

Try perplexity 5 and 100, and a different random_state. Do the four gestures still form groups?

CHALLENGE Bonus: run PCA first, then DBSCAN on the 2 PCA columns. Does it still find the groups?

Thank You

Keep learning, keep building



VSP TECHVERSE
An Institute of AI & Robotics